Lab 05: Complexity Hell

Zakamarki złożoności cyklomatycznej

Jarosław Hryszko
jaroslaw.hryszko@uj.edu.pl

Instytut Informatyki i Matematyki Komputerowej UJ

Otwarte repozytoria kodu i pomiary oprogramowania

Na co to komu?

Liczycie linie kodu. Fajnie.

Ale 50 linii prostych przypisań to nie to samo
co 20 linii zagnieżdżonych `if-else`.

LOC mówi **ile** kodu jest.
Złożoność cyklomatyczna mówi **jak bardzo boli**.

Wyobraźcie sobie

Jest 3 w nocy. Produkcja leży. Otwieracie plik i widzicie:

```
def process(data, mode, flag, ctx, retry=False):
    if mode == "A":
        if flag and not ctx.expired:
            if data.valid or (retry and data.partial):
                if ctx.role == "admin" or ctx.override:
                    ...
```

Ile ścieżek przez tę funkcję? Ile testów potrzeba?
Ile kubków kawy, żeby to zrozumieć?

Złożoność cyklomatyczna odpowiada na dwa pierwsze pytania.
Na trzecie niestety nie.

CC = ile ścieżek przez kod

```
def add(a, b):
    return a + b
```

$$CC = 1$$

Jeden test wystarczy.

```
def classify(x):
    if x > 0:
        if x > 100:
            return "big"
        return "positive"
    elif x == 0:
        return "zero"
    return "negative"
```

$$CC = 4$$

Minimum 4 testy na pełne pokrycie.

CC to **dolna granica** liczby testów potrzebnych do pokrycia wszystkich ścieżek.

Dlaczego to ma znaczenie w praktyce?

- **Code review:** „Ta funkcja ma CC = 47” jest lepszym argumentem niż „wydaje mi się, że jest za skomplikowana”
- **Testowanie:** Funkcja z CC = 25 wymaga minimum 25 testów. Ile z nich napiszecie? Pewnie 3. I potem się dziwicie.
- **Predykcja defektów:** Badania pokazują korelację między CC a liczbą bugów. Będziemy to weryfikować na labach 10-12.
- **Refaktoryzacja:** CC $>$ 10? Warto się przyjrzeć. CC $>$ 20? Warto się przerazić.

Metryka to nie wyrok

Funkcja z CC = 25 to niekoniecznie zły kod.

Dispatcher z 25 prostymi `case`'ami jest czytelniejszy
niż sprytna rekurencja z CC = 5, której nikt nie rozumie.

Metryka to **sygnał dymny**.

Nie mówi „napraw to". Mówi „tu warto zajrzeć".
A potem *człowiek* decyduje, czy jest problem.

Fun fact

Jądro Linuksa zawiera ponad 800 funkcji z CC > 50.
Rekord: `cache_alloc()` z CC $= 817$.

369 funkcji miało CC $>= 100$.
Nikt ich nie refaktoryzował, bo działają
i nikt nie jest pewien co się stanie po zmianach.

Niektóre funkcje przeżywają nie dlatego, że są dobre,
ale dlatego, że wszyscy się ich boją.

Źródło: Jbara, Matan, Feitelson, *High-MCC Functions in the Linux Kernel*,
Empirical Software Engineering, 2014. DOI: 10.1007/s10664-013-9275-7

Do roboty!

Instrukcja: `README.md` w repozytorium

„Cała Polska na ciebie patrzy, a ty się zaplątałaś w ten kod.” - Porucznik Borewicz